



3.5 自下而上语法分析

自上而下分析

- 方法

推导，从根到叶子构造语法树，从文法开始符产生句子。

- 优点

(1)对于产生语言来讲，自上而下分析是自然的，易于理解；

(2)分析表构造较简单。

- 缺点

(1)对文法要求严格——LL(k)文法；

(2) LL(k)文法难懂、难写、适用范围有限。



3.5 自下而上语法分析

自下而上(移进-归约)分析

- 方法

归约，从叶子到根构造语法树，从句子归约出文法开始符。

- 优点

(1)对于**分析语言**来讲，自下而上分析是自然的，因为语法分析处理的对象一开始都是终结符组成的输入序列，而不是文法开始符；

(2) 自下而上分析的最一般方法——LR方法，能力比自上而下分析的LL方法强，适用的文法更广泛，比较实用。

- 缺点

自下而上分析采用的逆向思维过程，使分析表构造复杂。对于规模稍大的语言，LR分析器的构造一般都借助于分析器生成工具，当前最广泛的工具之一是Yacc系列。



3.5 自下而上语法分析

本节内容（**本章重点**）

(1) 自下而上语法分析的一般方法和基本问题

- 概念：短语、直接短语、句柄、规范归约、剪句柄
- 移进-归约分析过程、符号栈的使用

(2) 自下而上分析的最一般方法——LR分析

- 驱动器算法
- 分析表构造
 - 概念：活前缀、LR(0)项目、LR(0)项目集规范族、有效项目
 - LR(0)分析表
 - SLR(1)分析表



3.5 自下而上语法分析

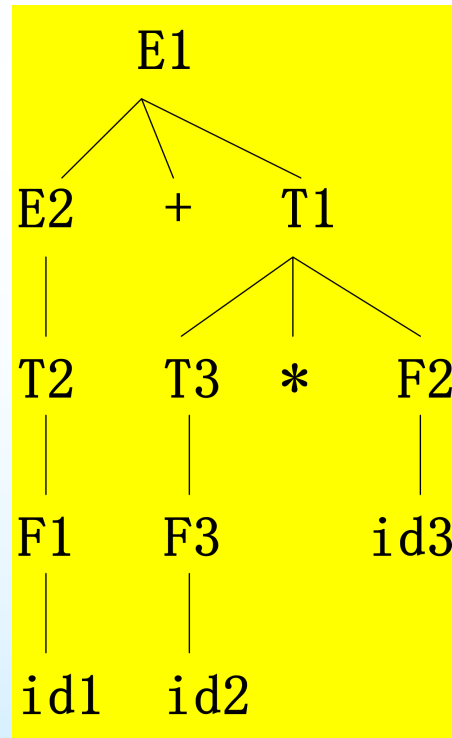
3.5.1 自下而上分析的基本方法

思路：从左到右扫描输入序列 ω ，经过一系列的步骤，最终将 ω 归约为文法开始符，或发现一个语法错误。**归约**，推导的逆过程，是一个反复用产生式的左部替换输入序列 ω 中和产生式的右部相同的子串、谋求对 ω 进行匹配的过程。

- 何时归约？
- 选用哪个产生式归约？

输入序列：id1+id2*id3

↑
ip



最左归约
(规范归约)

id1+id2*id3
≤ F1+id2*id3
≤ T2+id2*id3
≤ E2+id2*id3
≤ E2+F3*id3
≤ E2+T3*id3
≤ E2+T3*F2
≤ E2+T1
≤ E1

↑
最右推导
(规范推导)



3.5 自下而上语法分析

3.5.1 自下而上分析的基本方法

思路：从左到右扫描输入序列 ω ，经过一系列的步骤，最终将 ω 归约为文法开始符，或发现一个语法错误。

归约，推导的逆过程，是一个反复用产生式的左部替换产生式的右部、谋求对 ω 进行匹配的过程。

文法 (1) $S \rightarrow aABe$ (2) $A \rightarrow b$ (3) $A \rightarrow Abc$ (4) $B \rightarrow d$
句子： abbcde



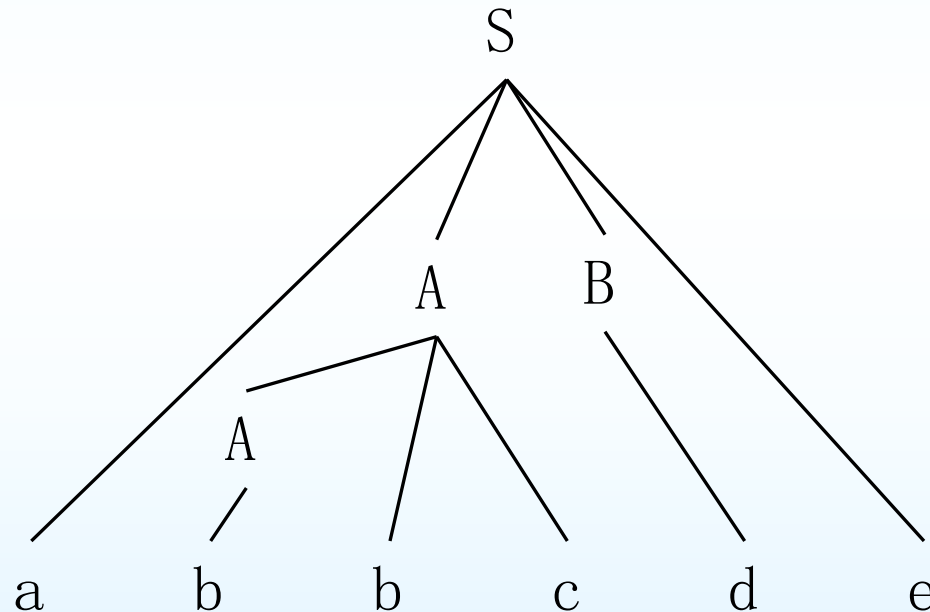
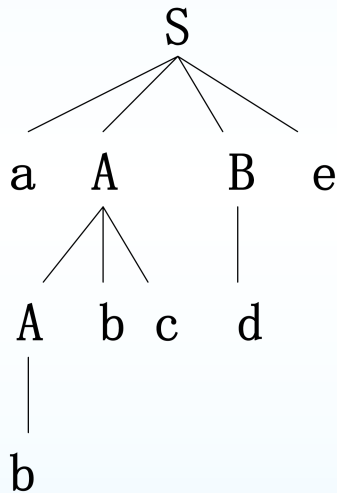
3.5 自下而上语法分析

文法 (1) $S \rightarrow aABe$ (2) $A \rightarrow b$ (3) $A \rightarrow Abc$ (4) $B \rightarrow d$

句子: abcde

从叶子到根构造语法树的过程

(每一步归约都是在构造一棵子树, 生成子树的根节点)



(2) $A \rightarrow b$

(3) $A \rightarrow Abc$

(4) $B \rightarrow d$

(1) $S \rightarrow aABe$



步骤	已扫描的前缀	栈(右端栈顶)	剩余输入	动作	右句型
(1)		#	abbcede#	移进	abbcede
(2)	a	#a	bbcede#	移进	abbcede
(3)	ab	#ab	bcde#	按 $A \rightarrow b$ 归约	abbcede
(4)	ab	#aA	bcde#	移进	aAbcede
(5)	abb	#aAb	cde#	移进	aAbcede
(6)	abbc	#aAbc	de#	按 $A \rightarrow Abc$ 归约	aAbcede
(7)	abbc	#aA	de#	移进	aAde
(8)	abbcd	#aAd	e#	按 $B \rightarrow d$ 归约	aAde
(9)	abbcd	#aAB	e#	移进	aABe
(10)	abbcede	#aABe	#	按 $S \rightarrow aABe$ 归约	aABe
(11)	abbcede	#S	#	接受	S

在输入序列语法结构正确的情况下，在语法分析的任意时刻，

- (1) 栈中的符号串+剩余输入串组成一个右句型；
- (2) 将已扫描的前缀部分归约为栈中的文法符号序列；
- (3) 从(1)~(11)，右句型的变化过程是abbcede最右推导的逆过程
 $abbcede \leq aAbcede \leq aAde \leq aABe \leq S$



步骤	已扫描的前缀	栈(右端栈顶)	剩余输入	动作	右句型
(1)		#	abbcede#	移进	abbcede
(2)	a	#a	bbcede#	移进	abbcede
(3)	ab	#ab	bcde#	按 $A \rightarrow b$ 归约	abbcede
(4)	ab	#aA	bcde#	移进	aAbcede
(5)	abb	#aAb	cde#	移进	aAbcde
(6)	abbc	#aAbc	de#	按 $A \rightarrow Abc$ 归约	aAbcde
(7)	abbc	#aA	de#	移进	aAde
(8)	abbcd	#aAd	e#	按 $B \rightarrow d$ 归约	aAde
(9)	abbcd	#aAB	e#	移进	aABe
(10)	abbcede	#aABe	#	按 $S \rightarrow aABe$ 归约	aABe
(11)	abbcede	#S	#	接受	S

文法 (1) $S \rightarrow aABe$ (2) $A \rightarrow b$ (3) $A \rightarrow Abc$ (4) $B \rightarrow d$

问题:

(1) 何时移进? 何时归约? 第(5)步为什么不将b归约为A?

(2) 用哪个产生式归约? ——

待归约的子串是什么? 归约成哪个非终结符?



步骤	已扫描的前缀	栈(右端栈顶)	剩余输入	动作	右句型
(1)		#	abbcde#	移进	abbcde
(2)	a	#a	bbcde#	移进	abbcde
(3)	ab	#ab	bcde#	按 $A \rightarrow b$ 归约	abbcde
(4)	ab	#aA	bcde#	移进	aAbcde
(5)	abb	#aAb	cde#	按 $A \rightarrow Ab$ 归约	aAbcde
(6)	abbc	#aA	cde#	移进	aAcde
(7)	abbc	#aAc	de#	移进	aAcde
(8)	abbcd	#aAcd	e#	按 $B \rightarrow d$ 归约	aAcde
(9)	abbcd	#aAcB	e#	移进	aAcBe
(10)	abbcde	#aAcBe	#	按 $S \rightarrow aAcBe$ 归约	aAcBe
(11)	abbcde	#S	#	接受	S

文法 (1) $S \rightarrow aAcBe$ (2) $A \rightarrow b$ (3) $A \rightarrow Ab$ (4) $B \rightarrow d$

句子: abbcde

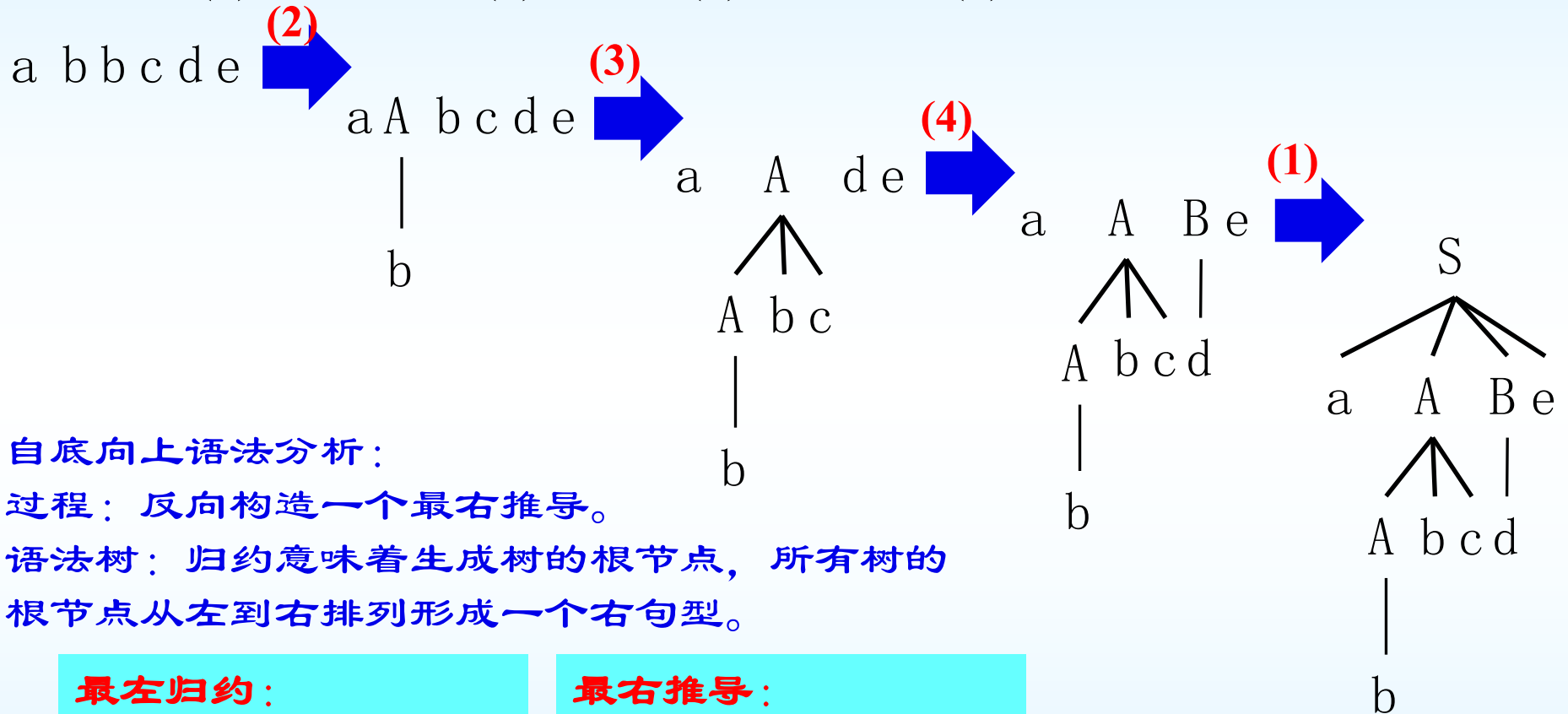
问题: 第(5)步, 是将b归约为A还是将Ab归约为A?

有些情况下, 文法中有右部相同的产生式, 例如, $A \rightarrow \alpha$ 和 $B \rightarrow \alpha$, 当栈顶形成 α 时, 选择哪个产生式归约?



3.5 自下而上语法分析

文法 (1) $S \rightarrow aABe$ (2) $A \rightarrow b$ (3) $A \rightarrow Abc$ (4) $B \rightarrow d$ 句子: abcde



自底向上语法分析:

过程: 反向构造一个最右推导。

语法树: 归约意味着生成树的根节点, 所有树的根节点从左到右排列形成一个右句型。

最左归约:

$S \leq aABe$	by (1)
$\leq aAde$	by (4)
$\leq aAbcde$	by (3)
$\leq abbcde$	by (2)

最右推导:

$S \Rightarrow aABe$	by (1)
$\Rightarrow aAde$	by (4)
$\Rightarrow aAbcde$	by (3)
$\Rightarrow abbcde$	by (2)



3.5 自下而上语法分析

1. 规范归约与“剪句柄”

定义3.13 设 $\alpha\beta\delta$ 是文法 G 的一个句型，若存在 $S \xRightarrow{*} \alpha A \delta$ ， $A \xRightarrow{+} \beta$ ，则称 β 是句型 $\alpha\beta\delta$ (相对于 A)的**短语**。特别的，若有 $A \rightarrow \beta$ ，则称 β 是句型 $\alpha\beta\delta$ (相对于产生式 $A \rightarrow \beta$)的**直接短语**。一个句型的最左直接短语被称为**句柄**。

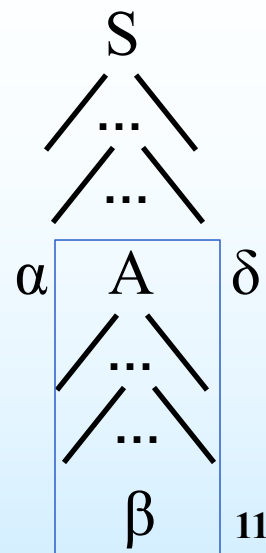
直观上，句型是一个完整结构，短语是句型中针对某非终结符的局部。因此，开始符号 S 是句型而不是短语。

短语形成的两个要素：

- ① 从 S 可以**推导**出含 A 的句型，即 $S \xRightarrow{*} \alpha A \delta$ ；
- ② 从 A **至少一次**推导出 β ，即 $A \xRightarrow{+} \beta$ 。

注意：

- 短语、直接短语、句柄都是针对句型而言的。只有选定“句型”，才能谈及短语、直接短语和句柄。
- 它们都是该句型的“子串”，是句型中的一部分。





3.5 自下而上语法分析

[例3.25] 从语法树角度观察

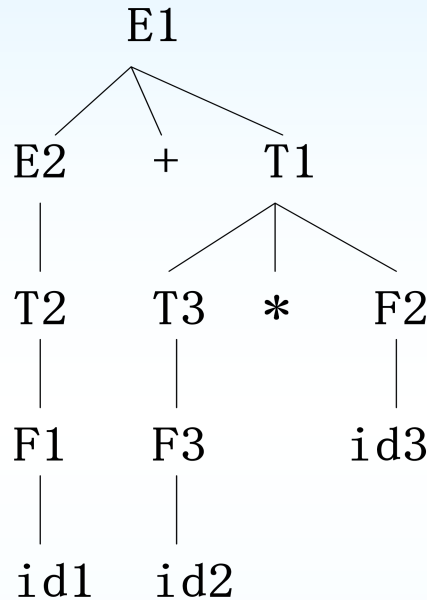
文法: $E \rightarrow E + T \mid T$

$T \rightarrow T * F \mid F$

$F \rightarrow \text{id}$

句型: $\text{id1} + \text{id2} * \text{id3}$

语法树:



句型 $\text{id1} + \text{id2} * \text{id3}$ 的

短语: $\text{id1} + \text{id2} * \text{id3}$ (E1)

$\text{id2} * \text{id3}$ (T1)

id1 (E2, T2, F1)

id2 (T3, F3)

id3 (F2)

直接短语: id1 (F1)

id2 (F3)

id3 (F2)

句柄: id1 (F1)

在句型的语法树中, 该句型的

短语: 以非终结符为根的子树中所有从左到右排列的**叶子**

直接短语: 只有父子关系的子树中所有从左到右排列的**叶子**(树高为2)

句柄: 最左边父子关系子树中所有从左到右排列的**叶子**(给定语法树, 则**句柄是唯一的**)

问题: $\text{id1} + \text{id2}$ 是 $\text{id1} + \text{id2} * \text{id3}$ 的短语吗? $T3 * F2$ 是谁的直接短语?



3.5 自下而上语法分析

定义3.14 若 α 是文法 G 的句子且满足下述条件, 则称序列 $\alpha_n, \alpha_{n-1}, \dots, \alpha_0$ 是 α 的一个**最左归约**:

1. $\alpha_n = \alpha$
2. $\alpha_0 = S$ (S 是 G 的开始符号)
3. 对任何 $i (0 < i \leq n)$, α_{i-1} 是将 α_i 中**句柄**替换为相应产生式左部非终结符得到的。

最左归约的逆过程是一个最右推导, 分别称最右推导和最左归约为**规范推导**和**规范归约**。

讨论:

- (1) **右句型的句柄之后不含非终结符。**
- (2) **如果 α_i 是右句型且 α_{i-1} 是将 α_i 的句柄替换为相应产生式左部非终结符得到的, 则 α_{i-1} 也是右句型且 α_i 是由 α_{i-1} 经最右推导得到的。**

由 (1) (2) 可得: 最左归约的逆过程是一个最右推导。



3.5 自下而上语法分析

(1) 右句型的句柄之后不含非终结符。

即：若 $\alpha\beta\delta$ 是右句型且 β 是句柄，则 $\delta \in T^*$ 。

证：令推导出 β 的非终结符是 A ，即有产生式 $A \rightarrow \beta$ 。由于 $\alpha\beta\delta$ 是右句型，所以，在 $\alpha\beta\delta$ 的最右推导过程中，子串 β 的生成（即对非终结符 A 的推导）也应符合最右推导，即，在推导 A 时， A 应是符号序列的最右非终结符，所以， δ 中不含非终结符。

(2) 如果 α_i 是右句型且 α_{i-1} 是将 α_i 的句柄替换为相应产生式左部非终结符得到的，则 α_{i-1} 也是右句型且 α_i 是由 α_{i-1} 经最右推导得到的。

证：令 $\alpha_i = \alpha\beta z$ ，其中 $z \in T^*$ ， β 是句柄，有产生式 $A \rightarrow \beta$ ， $\alpha_{i-1} = \alpha A z$ 。

● 反证：假设 $\alpha A z$ 不是右句型。根据假设可得， $\alpha A z$ 不存在最右推导。即，在 $\alpha A z$ 的推导过程中，一定会违反“每次展开最右边非终结符”的规定，即，在 α 中存在子串 γ_2 ($\alpha = \gamma_1 \gamma_2 \gamma_3$)可由某非终结符 B 推导得到 ($B \rightarrow \gamma_2$)，与“ β 是句柄”矛盾。所以， α_{i-1} 是右句型。

● 因为 A 是 α_{i-1} 中的最右非终结符。所以， $\alpha\beta z$ 是由 $\alpha A z$ 经最右推导得到的。



3.5 自下而上语法分析

定义3.14 若 α 是文法 G 的句子且满足下述条件，则称序列 $\alpha_n, \alpha_{n-1}, \dots, \alpha_0$ 是 α 的一个**最左归约**：

1. $\alpha_n = \alpha$
2. $\alpha_0 = S$ (S 是 G 的开始符号)
3. 对任何 $i (0 < i \leq n)$ ， α_{i-1} 是将 α_i 中**句柄**替换为相应产生式左部非终结符得到的。

最左归约的逆过程是一个最右推导，分别称最右推导和最左归约为**规范推导**和**规范归约**。

[例3.26] 文法 (1) $S \rightarrow aABe$ (2) $A \rightarrow b$ (3) $A \rightarrow Abc$ (4) $B \rightarrow d$

(2) (3) (4) (1)

$abbcde \leq aAbcde \leq aAde \leq aABe \leq S$

最左归约可以看成是从叶子到根构造语法树的过程，也可以看成是在假想的语法树上“剪句柄”的过程。



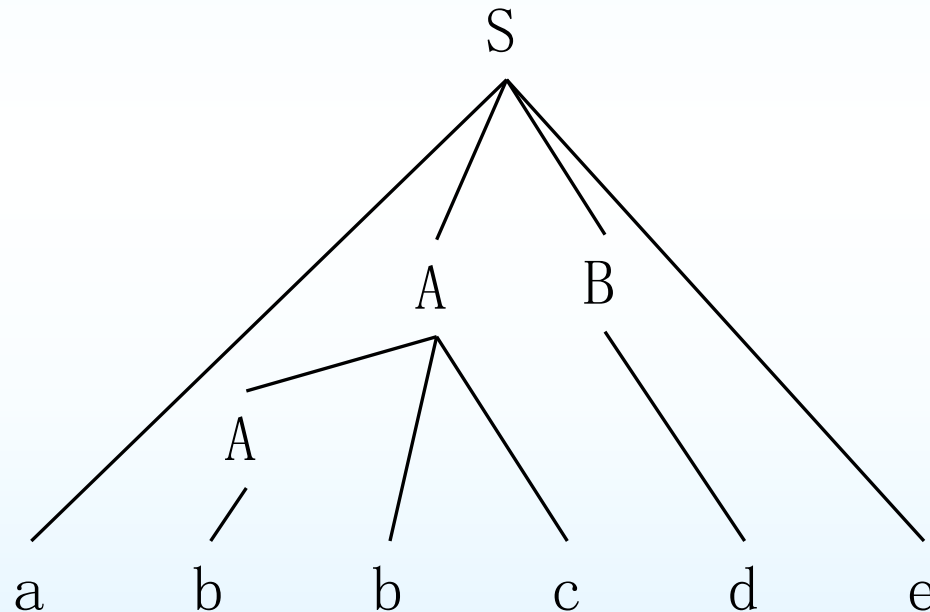
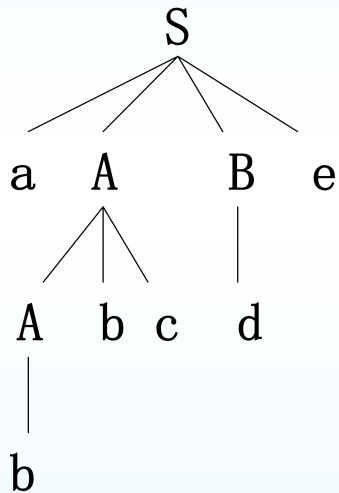
3.5 自下而上语法分析

文法 (1) $S \rightarrow aABe$ (2) $A \rightarrow b$ (3) $A \rightarrow Abc$ (4) $B \rightarrow d$

句子: abcde

从叶子到根构造语法树的过程

(每一步归约都是在构造一棵子树, 生成子树的根节点)



(2) $A \rightarrow b$

(3) $A \rightarrow Abc$

(4) $B \rightarrow d$

(1) $S \rightarrow aABe$



3.5 自下而上语法分析

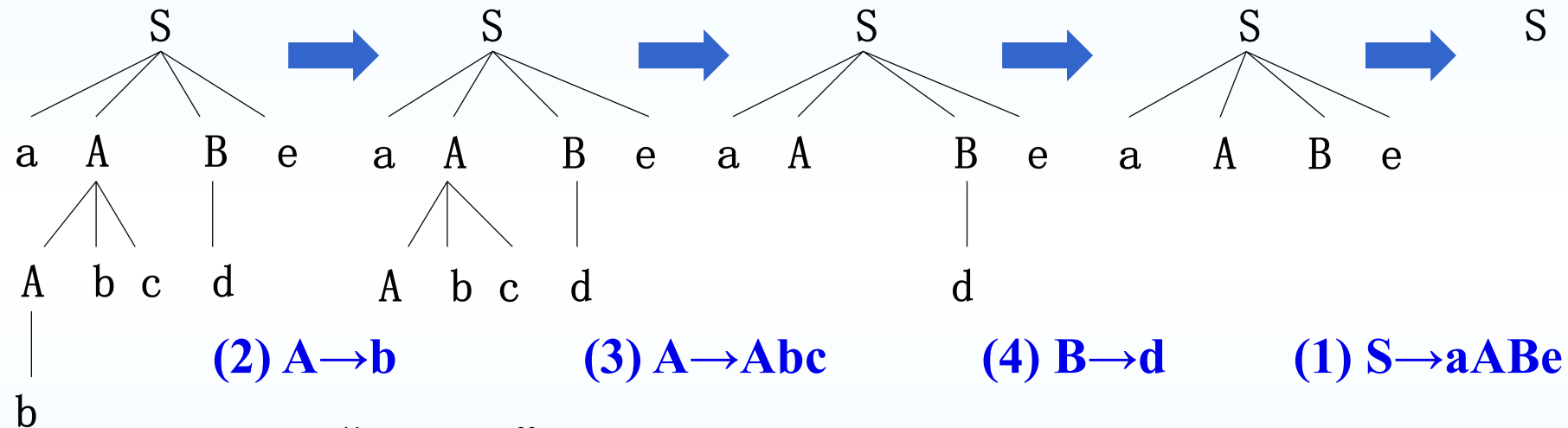
最左归约——“剪句柄”过程

$abbcde \leq aAbcde \leq aAde \leq aABe \leq S$

文法(1) $S \rightarrow aABe$ (2) $A \rightarrow b$ (3) $A \rightarrow Abc$ (4) $B \rightarrow d$

句子: $abbcde$

假设已经有了句子的语法树, 则:



用程序实现“剪句柄”，需要解决的问题：

- ① 确定右句型中将要归约的子串（**确定句柄**）；
- ② 确定如何选择正确的产生式进行归约。

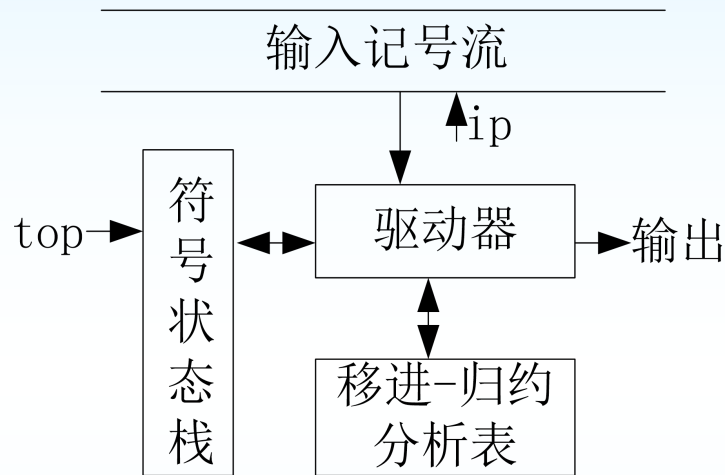
移进-归约：用一个**栈**“记住”将要包含句柄的**前缀**；用**分析表**确定何时栈顶已形成句柄以及归约用到的产生式。

句柄形成前，移进终结符；形成后，归约为非终结符。

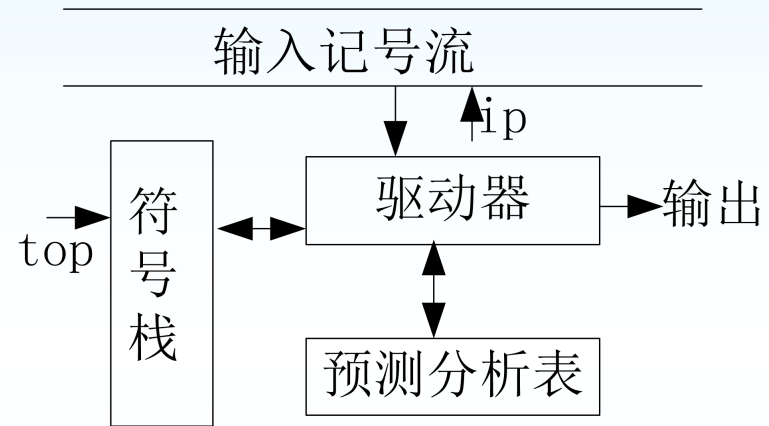


3.5 自下而上语法分析

2. 移进-归约分析器工作模式



(a) 移进-归约分析器模型



(b) 预测分析器模型

移进-归约分析器:

1. 分析方法: 格局与格局变换
2. 分析表
3. 驱动器 (模拟算法)
4. LR (文法、语言、分析器)
5. LR(0)和SLR(1)分析表的构造

预测分析器:

1. 分析方法: 格局与格局变换
2. 分析表
3. 驱动器 (模拟算法)
4. LL (文法、语言、分析器)
5. 预测分析表的构造

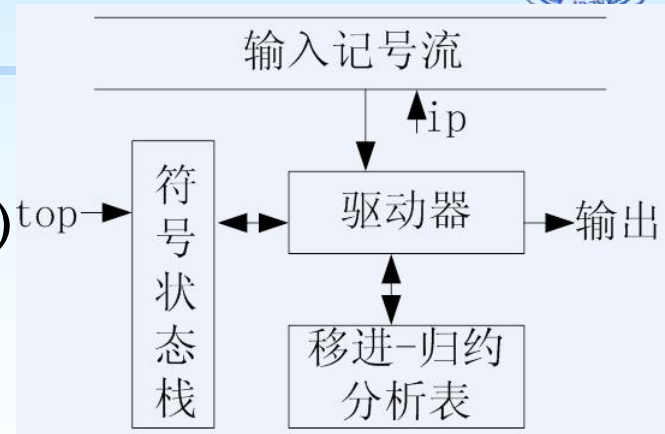


3.5 自下而上语法分析

工作方法：放幻灯，每个幻灯片是一个格局。

格局：（#栈，当前剩余输入#，改变格局的动作）

- **剩余输入：**开始格局时为全部输入序列；
接收格局时为空；其他格局时为后缀。
- **栈：**存放文法符号和状态，并且符号和状态成对出现。
- **动作：**驱动器根据当前栈和剩余输入，查找分析表确定相应动作，
改变栈和剩余输入的内容，进入下一格局。



改变格局的动作：

- ① 移进(shift)：输入序列中的终结符进栈。(当栈顶没有形成句柄时)
- ② 归约(reduce)：将栈顶句柄替换为对应非终结符。(当栈顶形成句柄时)
- ③ 接受(accept)：宣告分析成功。
- ④ 报错(error)：发现语法错误，调用错误恢复例程。

3.5 自下而上语法分析

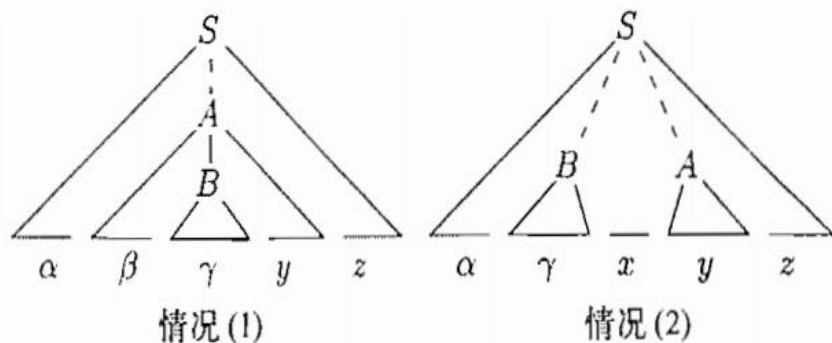
注意：

1. 句柄总是在栈顶形成（最左归约）：
语法分析器在进行一次归约之后，
都必须接着移进零个或多个符号才
能在栈顶找到下一个句柄。

2. 栈中总是含有一个右句型的前缀且
该前缀不包含句柄之后的符号——活前缀。

LR分析是如何解决这两个问题的？

- 1、如何判断栈顶是否形成句柄？
- 2、当句柄形成时，如何选择产生式？



[例3.27] 用移进-归约方法分析abbcde：

abbcde $\leq$ aAbcde $\leq$ aAde $\leq$ aABe $\leq$ S

栈	剩余输入	改变格局的动作
#	abbcde#	移进
#a	bbcd#	移进
#ab	bcde#	归约，(2) $A \rightarrow b$
#aA	bcde#	移进
#aAb	cde#	移进
#aAbc	de#	归约，(3) $A \rightarrow Abc$
#aA	de#	移进
#aAd	e#	归约，(4) $B \rightarrow d$
#aAB	e#	移进
#aABe	#	归约，(1) $S \rightarrow aABe$
#S	#	接受

考虑任意最右推导中的两个连续步骤，假设 $x, y, z \in T^*$, $\alpha, \beta, \gamma \in (N \cup T)^*$, $A, B \in N$

情况(1) $S \Rightarrow^* \alpha A z \Rightarrow^* \alpha \beta B \gamma z \Rightarrow^* \alpha \beta \gamma \gamma z \Rightarrow^* \alpha \beta x_3 \gamma z \Rightarrow^* \alpha x_2 x_3 \gamma z \Rightarrow^* x_1 x_2 x_3 \gamma z$

情况(2) $S \Rightarrow^* \alpha B x A z \Rightarrow^* \alpha B x \gamma z \Rightarrow^* \alpha \gamma x \gamma z \Rightarrow^* \alpha x_2 x \gamma z \Rightarrow^* x_1 x_2 x \gamma z$



3.5 自下而上语法分析

3.5.2 LR分析

LR分析的特点：

- ① 采用最一般的无回溯移进—归约方法
- ② 可分析的文法是LL文法的真超集，适合几乎所有的程序设计语言
- ③ 能及时发现错误，快到从左到右扫描输入序列的最大可能
- ④ 分析表较复杂，难以手工构造

LR分析的核心：**LR分析表+驱动器**

内容：(1)工作原理(分析表的组成、分析算法)

(2)分析表的构造

讨论依据的文法：

$$E \rightarrow \mathbf{E - T} \mid T \quad (1) \quad (2)$$

$$T \rightarrow T * F \mid F \quad (3) \quad (4)$$

$$F \rightarrow \mathbf{-F} \mid \text{id} \quad (5) \quad (6)$$



3.5 自下而上语法分析

3.5.2 LR分析

1. LR分析与LR文法
2. LR(0)分析表的构造 (教材第3.3.4~3.3.7节)
 - 2.1 LR(0)项目和LR(0)项目集规范族
 - 2.2 LR(0)项目集规范族的构造
 - 2.3 有效项目
 - 2.4 LR(0)分析表的构造
3. SLR(1)分析表的构造 (教材第3.3.8节)
4. 非SLR(1)文法
5. 基于LR分析的语法分析器生成器简介



3.5 自下而上语法分析

3.5.2 LR分析

1. LR分析与LR文法

2. LR(0)分析表的构造 (教材第3.3.4~3.3.7节)

2.1 LR(0)项目和LR(0)项目集规范族

2.2 LR(0)项目集规范族的构造

2.3 有效项目

2.4 LR(0)分析表的构造

3. SLR(1)分析表的构造 (教材新书3.3.8节)

4. 非SLR(1)文法

5. 基于LR分析的语法分析器生成器简介



3.5 自下而上语法分析

$E \rightarrow E-T \mid T$

$T \rightarrow T * F \mid F$

$F \rightarrow -F \mid id$

1. LR分析与LR文法

分析表

	id	-	*	#	E	T	F
0	s4	s5			1	2	3
1		s6		acc			
2		r2	s7	r2			
3		r4	r4	r4			
4		r6	r6	r6			
5	s4	s5					8
6	s4	s5				9	3
7	s4	s5					10
8		r5	r5	r5			
9		r1	s7	r1			
10		r3	r3	r3			

动作表(action)

转移表(goto)提示: ②和⑤共同完成归约。

格局与动作(栈中文法符号和状态成对出现)

开始格局: ($\#0$, $\omega\#$, 移进?)

结束格局: ($\#0E1$, $\#$, 接收)

出错格局: ($\#\delta$, $\omega'\#$, 报错)

改变格局的动作:

① $action[s,a] = si$: 移进并转向i状态

② $= rj$: 用第j个产生式

的左部替换栈中的句柄

③ $= acc$: 接收

④ $=$ 空白: 报错

⑤ $goto[s,A]=s'$: 次栈顶s状态遇到栈顶A转移到状态s'

$action[s,a]$ 确定改变格局的动作(与栈顶状态s和当前输入终结符a有关)

$goto[s,A]$ 指示非终结符的状态转移(与栈顶非终结符A和次栈顶状态s有关)



3.5 自下而上语法分析

1. LR分析与LR文法

LR分析器的工作过程可以看作栈的内容和剩余输入序列所构成的二元组的变化过程。

分析开始时的二元组为 $(\#0, a_0a_1\dots a_n\#)$ ，设每一步的分析结果可表示为 $(\#0X_1s_1X_2s_2\dots X_ms_m, a_ia_{i+1}\dots a_n\#)$ 。

分析器的下一步动作是由当前输入符号 a_i 和栈顶状态 s_m 唯一决定。

查找分析表，

(1)如果 $\text{action}[s_m, a_i] = \text{移进 } s$ ，则分析器执行移进动作，进入格局 $(\#0X_1s_1X_2s_2\dots X_ms_m a_i s, a_{i+1}\dots a_n\#)$ ，分析器将当前输入 a_i 和下一状态 s 压入栈中。

s
a_i
s_m
X_m
...
s_1
X_1
0
#



3.5 自下而上语法分析

(2)如果 $\text{action}[s_m, a_i] = \text{归约 } A \rightarrow \beta$ ，则分析器执行归约动作，进入格局 $(\#0X_1s_1X_2s_2\ldots X_{m-|\beta|}s_{m-|\beta|}As, a_ia_{i+1}\ldots a_n\#)$ 。其中， $s = \text{goto}[s_{m-|\beta|}, A]$ 。

分析器首先从栈中弹出 $2|\beta|$ 个符号，包含 $|\beta|$ 个状态符号和 $|\beta|$ 个文法符号，这些文法符号刚好匹配产生式右部的 β ，这时暴露出状态 $s_{m-|\beta|}$ ；

然后，把产生式左部的符号 A 压入栈中；

最后，将 $\text{goto}[s_{m-|\beta|}, A]$ 状态 s 压入栈中。

注意：(1)归约动作和goto表配合使用：每执行完归约动作，都需查找goto表生成与非终结符配对的状态。

(2)执行归约动作，当前剩余输入不发生变化。

s_m
s
A
$s_{m- \beta }$
$X_{m- \beta }$
$\dots$
s_1
X_1
0
$\#$



算法3.8 LR分析

输入 输入序列 ω 和文法 G 的LR分析表 (action与goto)

输出 若 ω 属于 $L(G)$, 得到 ω 的规范归约, 否则指出一个错误

方法 初始格局为: $(\#0, \omega\#, ?)$, 其中0是初态

ip指向 $\omega\#$ 中的第一个终结符, top指向栈顶初始状态;

loop $s := \text{top}^\wedge$; $a := \text{ip}^\wedge$;

case **action**[s , a] is

shift s' : push(a); push(s'); next(ip); -- 移进

reduce by $A \rightarrow \beta$:

pop($2 * |\beta|$); -- 弹出句柄和相应状态

$s' := \text{top}^\wedge$; -- 暴露出当前栈顶状态 s'

push(A); -- 产生式左部符号进栈

push(**goto**(s' , A)); -- 新栈顶状态进栈

write($A \rightarrow \beta$); -- 完成归约, 跟踪分析轨迹

accept: return; -- 成功返回

others: error; -- 出错处理

end case;

end loop;

实际的算法中仅存放状态, 而在分析的格局中, 仅显示文法符号。



3.5 自下而上语法分析

	id	-	*	#	E	T	F
0	s4	s5			1	2	3
1		s6		acc			
2		r2	s7	r2			
3		r4	r4	r4			
4		r6	r6	r6			
5	s4	s5					8
6	s4	s5				9	3
7	s4	s5					10
8		r5	r5	r5			
9		r1	s7	r1			
10		r3	r3	r3			

shift s': push(a); push(s'); next(ip);
 reduce by $A \rightarrow \beta$:
 pop($2 * |\beta|$); s' := top[^]; push(A);
 push(goto(s', A));
 write($A \rightarrow \beta$);

栈	剩余输入	动作
#0	id--id*id#	s4
#0id4	--id*id#	r6($F \rightarrow id$)
#0F3	--id*id#	r4($T \rightarrow F$)
#0T2	--id*id#	r2($E \rightarrow T$)
#0E1	--id*id#	s6
#0E1-6	-id*id#	s5
#0E1-6-5	id*id#	s4
#0E1-6-5id4	*id#	r6($F \rightarrow id$)
#0E1-6-5F8	*id#	r5($F \rightarrow -F$)
#0E1-6F3	*id#	r4($T \rightarrow F$)
#0E1-6T9	*id#	s7
#0E1-6T9*7	id#	s4
#0E1-6T9*7id4	#	r6($F \rightarrow id$)
#0E1-6T9*7F10	#	r3($T \rightarrow T * F$)
#0E1-6T9	#	r1($E \rightarrow E - T$)
#0E1	#	acc



3.5 自下而上语法分析

定义3.15 若为文法G构造的移进—归约分析表中不含多重定义的条目，则称G为**LR(k)文法**，分析器被称为是**LR(k)分析器**，它所识别的语言被称为**LR(k)语言**。**L表示从左到右扫描输入序列，R表示逆序的最右推导，k表示为确定下一动作向前看的终结符个数**，即当在栈顶看到某产生式的右部时，需要再向后扫描输入序列k个符号，才能决定是否用该产生式归约。一般情况下 $k \leq 1$ 。当 $k=1$ 时，简称LR。

LR分析器是一类分析器

1. 根据分析表的构造，有LR(0)、SLR(1)、LALR(1)和LR(1)分析器。
2. 它们功能的强度和构造的难度依次递增。
3. 当 $k > 1$ 后，分析器的构造趋于复杂，一般情况下并不构造 $k > 1$ 的LR(k)分析器。
4. 我们构造LR(0)和SLR(1)分析器。